Search, Games, and MDPs

From Optimal Paths to Sequential Decisions

1

Learning Objectives

- Apply [**A* search**](https://oakland0-my.sharepoint.com/:p:/g/personal/tianlema_oakland_edu/IQDIjtgAcT8zRp8Cq1ClmBdhAUe1UWwqB4gEneUczEnHGvc?e=FCy6sX) with an admissible heuristic to find optimal paths
- Trace **minimax** evaluation and **alpha-beta pruning** on a game tree
- Explain **MCTS** and the exploration-exploitation tradeoff (**UCB1**)
- Define a **Markov Decision Process** and write the **Bellman equation**
- Formulate **LLM text generation** as an MDP

2

The Big Picture

- Your transformer already does search — **beam search** picks the highest-scoring sequence from an exponentially large space
- Today we formalize what **search** means, see where it breaks, and build the **mathematical framework** for everything in this unit
- **The arc**: Search (known rules) → RL (unknown rules) → LLM training (the frontier)

3

Part 1: Informed Search — A*

4

Search as Problem Solving

- A **search problem** has five components:
- 1. **Initial state** — where we start
- 2. **Actions** — what moves are available
- 3. **Transition model** — what happens when we take an action
- 4. **Goal test** — have we reached the destination?
- 5. **Path cost** — how expensive is the path so far?
- Example: finding the **shortest route** in a weighted road network

5

From Uninformed to Informed Search

- **Uninformed** strategies know nothing about how close a state is to the goal:
- **BFS**: finds shallowest goal — ignores edge weights, explores in all directions
- **DFS**: memory-efficient but no optimality guarantee, can get lost in deep branches
- **Uniform-Cost Search (UCS)**: expands cheapest-$g(n)$ node — optimal, but explores a growing **circle** around the start
- All three can waste enormous time exploring states **away from the goal**
- **Informed search** adds a **heuristic** $h(n)$ — an estimate of remaining cost — to steer search **toward** the goal

6

A* Evaluation Function

- A* uses:

$$f(n) = g(n) + h(n)$$

- $g(n)$: **actual cost** from start to node n
- $h(n)$: **estimated cost** from n to goal (the heuristic)
- A* always expands the node with the **lowest** $f(n)$ from the priority queue
- Combines the best of **uniform-cost search** (optimal) and **greedy search** (fast)

7

Admissibility and Consistency

- **Admissibility**: $h(n) \leq h^*(n)$ — never overestimates the true cost
- Guarantees A* finds the **optimal path**
- **Consistency** (stronger): $h(n) \leq c(n, n') + h(n')$ for every successor n'
- Guarantees each node is expanded **at most once**
- Example heuristics: **straight-line distance** for maps, **Manhattan distance** for grids
- An **inadmissible** heuristic can still be useful — finds good solutions fast, just not provably optimal

8

A* Search Algorithm

```
function AStar(start, goal, h)
  OPEN ← priority queue with {start}, g(start) = 0
  CLOSED ← empty set

  while OPEN is not empty do
    n ← node in OPEN with lowest f(n) = g(n) + h(n)
    if n = goal then
      return reconstruct_path(n)
    move n to CLOSED
    for each successor m of n do
      tentative_g ← g(n) + cost(n, m)
      if tentative_g < g(m) then
        g(m) ← tentative_g
        parent(m) ← n
        add or update m in OPEN
  return failure  // no path exists
```

9

A* Worked Example

- Graph: S→A (1), S→B (4), A→B (2), A→G (10), B→G (5). Heuristic: h(S)=7, h(A)=6, h(B)=4, h(G)=0
- **Step 1**: Expand S. f(A)=1+6=**7**, f(B)=4+4=**8**. OPEN: {A(7), B(8)}
- **Step 2**: Expand A (lowest f). f(B via A)=1+2+4=**7** < 8 → update B. f(G via A)=1+10+0=**11**. OPEN: {B(7), G(11)}
- **Step 3**: Expand B. f(G via B)=3+5+0=**8** < 11 → update G. OPEN: {G(8)}
- **Step 4**: Expand G — **goal reached!** Optimal path: S→A→B→G, cost = **8**
- Notice: A* never expanded any node with $f >$ optimal cost

10

Interactive Demo: A* Algorithm

🚀 **Interactive Demo:** astar_demo.html

11

A* and LLM Decoding

- **Beam search** in transformer decoding is a form of search — keeps top-k candidates at each step
- But beam search has **no admissible heuristic** and **no optimality guarantee**
- It's memory-efficient but can miss the global optimum
- A* shows what's possible **when you have a good heuristic**
- A question for later: *what if a neural network could learn the heuristic?*

12

Part 2: Game Playing — Adversarial Search

13

Adversarial Search: The Challenge

- Search gets harder when an **opponent** is actively trying to beat you
- **Two-player zero-sum games**: one player's gain is the other's loss
- Examples: chess, Go, tic-tac-toe
- We need to plan assuming the opponent plays **optimally**

14

Game Trees

- A **game tree** encodes all possible plays from the current position
- Nodes alternate between **MAX** (our turn) and **MIN** (opponent's turn)
- **Leaf nodes** have a **utility** score: +1 (win), 0 (draw), −1 (loss) — or a richer evaluation
- **Branching factor** b and **depth** d determine tree size: $O(b^d)$
- Chess: $b \approx 35$, $d \approx 80 \rightarrow \sim 10^{120}$ nodes. **Exhaustive search is impossible.**
- We need a principled way to choose moves **without seeing the whole tree**

15

Minimax Algorithm

- Alternating **MAX** and **MIN** layers in the game tree:
- $$\text{minimax}(s) = \begin{cases} \text{utility}(s) & \text{if } s \text{ is terminal} \\ \max_a \text{minimax}(\text{result}(s,a)) & \text{if MAX's turn} \\ \min_a \text{minimax}(\text{result}(s,a)) & \text{if MIN's turn} \end{cases}$$
- MAX chooses the move that **maximizes** the minimum guaranteed outcome
- Optimal play for **both** sides — assumes a perfect opponent

16

Minimax Worked Example

- Consider a game tree with depth 2 (MAX→MIN→leaves):
- Left subtree leaves: [3, 5] — MIN picks **3**
- Middle subtree leaves: [2, 9] — MIN picks **2**
- Right subtree leaves: [7, 1] — MIN picks **1**
- MAX sees values [3, 2, 1] → picks the **left** branch with value **3**
- **Interpretation**: even though leaf 9 exists, MAX cannot reach it — MIN would block it
- Minimax returns the **guaranteed** outcome under optimal play by both sides

17

Depth-Limited Minimax

- In real games, we **cannot search to terminal states** — the tree is too deep
- **Solution**: cut off search at depth d and apply an **evaluation function** $\text{eval}(s)$
- $$\text{minimax}(s, d) = \begin{cases} \text{eval}(s) & \text{if } d = 0 \text{ or } s \text{ is terminal} \\ \max_a \text{minimax}(\text{result}(s, a),\ d-1) & \text{if MAX's turn} \\ \min_a \text{minimax}(\text{result}(s, a),\ d-1) & \text{if MIN's turn} \end{cases}$$
- The evaluation function is **domain knowledge** — e.g., material count in chess
- Quality of play depends entirely on the **quality of eval** and the **search depth**

18

Alpha-Beta Pruning: Intuition

- **Key insight**: you don't need to evaluate every branch to know the optimal move
- If MAX already has a move guaranteeing score **3**, and a new branch reveals MIN can force score **1** there, MAX will **never choose that branch** — stop exploring it
- We maintain two values during search:
- α = best score MAX can **guarantee** so far (MAX's lower bound)
- β = best score MIN can **guarantee** so far (MIN's upper bound)
- **Prune** whenever $\alpha \geq \beta$ — the remaining children cannot affect the outcome

19

Alpha-Beta Pruning: Mechanics

- At a **MAX node**: update $\alpha = \max(\alpha, \text{child value})$. If $\alpha \geq \beta$, prune remaining children (**beta cutoff**)
- At a **MIN node**: update $\beta = \min(\beta, \text{child value})$. If $\alpha \geq \beta$, prune remaining children (**alpha cutoff**)
- **Best case**: reduces branching factor from b to $\sqrt{b}$ — effectively **doubles search depth** for the same compute
- **Move ordering** matters: examining the best move first maximizes pruning
- Same result as full minimax — **never changes the answer**, just finds it faster

20

Alpha-Beta Worked Example

- Same tree: MAX→MIN→leaves. Left: [3, 5], Middle: [2, 9], Right: [7, 1]
- **Left subtree**: MIN returns 3. MAX updates $\alpha = 3$
- **Middle subtree**: MIN sees child 2. Since 2 < α (3), MAX would never pick this branch. MIN updates $\beta = 2$. Now $\alpha(3) \geq \beta(2)$ → **prune child 9** — never evaluated!
- **Right subtree**: MIN sees child 7, updates $\beta = 7$. Then sees child 1, updates $\beta = 1$. Since $\alpha(3) \geq \beta(1)$ — but we already evaluated both children here
- Result: same answer (**3**), but we skipped evaluating leaf 9
- With better move ordering, we could prune even more

21

The Limits of Minimax + Alpha-Beta

- Even with alpha-beta, the tree is still $O(b^{d/2})$ — exponential in depth
- Chess ($b \approx 35$): manageable with deep search + good eval → superhuman play since 1997
- Go ($b \approx 250$, $d \approx 150$): $\sim 10^{170}$ positions — **completely intractable**
- Writing a good evaluation function for Go is also **extremely hard**
- We need a fundamentally different approach: **sampling** instead of exhaustive search

22

Monte Carlo Tree Search (MCTS): Overview

- MCTS doesn't need a handcrafted evaluation function — it **estimates values by sampling**
- Build the search tree **incrementally** — focus effort on the most promising regions
- Each iteration has **four phases**: Selection → Expansion → Simulation → Backpropagation
- After many iterations, choose the most-visited child of the root
- Asymptotically converges to **minimax** — but useful results come **much faster**

23

MCTS Phase 1–2: Selection and Expansion

- **Selection**: starting from the root, repeatedly pick the child with the highest **UCB1 score** until we reach a node with unexplored children
- This descends the existing tree, balancing **exploitation** (high win rate) and **exploration** (few visits)
- **Expansion**: add **one new child** to the tree — the first unexplored action from the selected node
- The tree grows by exactly **one node per iteration**
- Over time, the tree becomes deeper in promising areas and shallow in poor ones

24

MCTS Phase 3–4: Simulation and Backpropagation

- **Simulation** (rollout): from the new node, play out to a terminal state using a **default policy**
- Simplest default policy: **random moves**. Better: lightweight heuristics or a **neural network**
- The outcome (win/loss/score) is the return from this one sample
- **Backpropagation**: walk back up the tree from the new node to the root
- At each ancestor: increment visit count N, add the result to the total reward Q
- This updates the **win rate estimates** that guide future selection

25

UCB1: Exploration vs. Exploitation

- The first formal appearance of the **exploration-exploitation tradeoff**:
- $$a^* = \arg\max_a \left[\underbrace{\bar{Q}(a)}_{\text{exploit}} + \underbrace{c\sqrt{\frac{\ln N}{n(a)}}}_{\text{explore}} \right]$$
- $\bar{Q}(a)$: average reward from action a so far (exploit what works)
- $\sqrt{\frac{\ln N}{n(a)}}$: **uncertainty bonus** — drives us toward under-explored options
- c: exploration constant that balances the two terms
- This same tradeoff reappears **throughout RL**

26

MCTS Worked Example

- Root has 3 actions: A (4 wins / 6 visits), B (1/1), C (2/5). Total $N = 12$. $c = \sqrt{2}$
- UCB1(A) = $4/6 + \sqrt{2} \cdot \sqrt{\ln 12/6}$ = 0.67 + 0.88 = **1.55**
- UCB1(B) = $1/1 + \sqrt{2} \cdot \sqrt{\ln 12/1}$ = 1.00 + 2.22 = **3.22** ← selected (barely tried!)
- UCB1(C) = $2/5 + \sqrt{2} \cdot \sqrt{\ln 12/5}$ = 0.40 + 0.99 = **1.39**
- B is selected despite a lower visit count — UCB1 **demands we explore uncertain options**
- After enough visits, B's exploration bonus shrinks and the best action emerges

27

Interactive Demo: MCTS

🚀 **Interactive Demo:** mcts_demo.html

28

From MCTS to AlphaGo

- **AlphaGo** (2016): replaced both key MCTS components with neural networks
- **Policy network** $p_\theta(a|s)$: guides selection — replaces uniform prior over moves
- **Value network** $v_\phi(s)$: evaluates positions — replaces random rollouts
- Result: superhuman Go play, defeating world champion Lee Sedol 4–1
- **AlphaZero** (2017): learned entirely from self-play — no human game data at all
- We'll see this pattern again: **MCTS + neural networks** for LLM reasoning and planning

29

Part 3: From Search to Learning — MDPs

30

The Pivot: Why We Need RL

- A* and minimax assume you **know the transition model** perfectly
- For most real problems — and for **training LLMs** — you face:
- ❌ **Unknown rules** — you don't know the environment dynamics
- ❌ **Stochastic outcomes** — actions have probabilistic effects
- ❌ **No goal state** — just scalar reward signals
- We need a framework for **learning good behavior from experience**

31

Search/Planning vs. Reinforcement Learning

	Classical Search	Reinforcement Learning
Model of world	Known exactly	Unknown — learned from interaction
Outcomes	Deterministic (typically)	Stochastic
Objective	Reach a goal state	Maximize cumulative reward
Approach	Compute optimal plan	Learn optimal behavior from experience

32

The MDP Formalism

- A **Markov Decision Process** is a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:
- $\mathcal{S}$: set of **states**
- $\mathcal{A}$: set of **actions**
- $P(s'|s,a)$: **transition probability** — probabilistic reasoning enters RL here
- $R(s,a,s')$: **reward function** — scalar feedback
- $\gamma \in [0,1)$: **discount factor** — how much we value future vs. immediate reward

33

Markov Property and Policies

- **Markov property**: $P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \ldots) = P(s_{t+1}|s_t, a_t)$
- The future depends **only on the present state and action**, not the full history
- This is what makes the problem mathematically **tractable**
- **Policy**: $\pi(a|s)$ — a probability distribution over actions given a state
- This is the object we want to **optimize**

34

The Key Connection to LLMs

- You already know what a stochastic policy looks like!
- The **softmax output** of a transformer over a 100K-token vocabulary:
- $$\pi_\theta(\text{token}|\text{context}) = \text{softmax}(\text{logits})$$
- **Your LLM is a policy.** You just didn't call it that.
- RL will teach us how to **improve this policy** using reward signals

35

Return and Value Functions

- **Return**: total discounted future reward from timestep t:
- $$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$
- **State-value function**: how good is this state?
- $$V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$$
- **Action-value function**: how good is this action in this state?
- $$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$$

36

The Bellman Equation

- The recursive structure that makes RL **mathematically possible**:
- $$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) \left[R(s,a,s') + \gamma V^\pi(s')\right]$$
- **In words**: the value of a state equals the expected immediate reward plus the discounted value of wherever you end up

37

Interactive Demo: Bellman Value Iteration

🚀 **Interactive Demo:** bellman_demo.html

38

LLM as an MDP

MDP Concept	LLM Equivalent
State s	Prompt + tokens generated so far
Action a	Next token from vocabulary (|V| ≈ 100K)
Policy $\pi_\theta(a\|s)$	Transformer's softmax output distribution
Transition $P(s'\|s,a)$	Deterministic: append chosen token to context
Reward R	Score at end of generation (human, reward model, or verifier)
Episode	One complete generated response

39

Summary

40

All Interactive Demos

🚀 **Interactive Demo:** astar_demo.html

🚀 **Interactive Demo:** mcts_demo.html

🚀 **Interactive Demo:** bellman_demo.html

41

Lecture Summary

- **A***: Optimal search with $f(n) = g(n) + h(n)$ — requires an admissible heuristic
- **Minimax + Alpha-Beta**: Optimal play in adversarial games — but doesn't scale to Go
- **MCTS + UCB1**: Sample-based search with exploration-exploitation — conquered Go, returns in LLM reasoning
- **MDPs**: The formal framework $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ for sequential decisions under uncertainty
- **Bellman Equation**: Value = immediate reward + discounted future value — the backbone of RL
- **Next**: We can't enumerate states or know transitions for LLMs — we need to *learn* from experience → policy gradients

42

Supplementary Resources

🚀 **Interactive Demo:** handout.html

- [Colab Notebook](https://colab.research.google.com/drive/1pt9rP1aO1gdbm7ahMKrSKqkydK_CqscX)

43